Kdapt: A Runtime-Aware Adaptive Bin-Packing Scheduler Framework Plugin for Kubernetes

Department of Computer Science and Electrical Engineering
University of Maryland, Baltimore County

CMSC 699 — Independent Study

Advisor — Konstantin Kalpakis

Spring 2026

Source Code Repository:
github.com/kvv1618/kubernetes

Varun Kesharaju

May 2026

Abstract

The default Kubernetes scheduler distributes pods across cluster nodes using a load-spreading heuristic that optimizes for availability but frequently fragments cluster resources, leaving many nodes partially utilized and preventing idle nodes from being freed for scale-down. This report presents **Kdapt**, a custom scheduler plugin that implements a *runtime-aware adaptive bin-packing* strategy built on top of the Kubernetes Scheduling Framework. Kdapt replaces the default spreading behavior with deliberate consolidation: pods are placed on the most-utilized feasible node, freeing quieter nodes and reducing fragmentation. The key novelty is the fusion of two complementary signals — static resource *requests* for future-aware safety and live *runtime metrics* for accuracy — into a single blended utilization signal using Exponential Moving Average (EMA) smoothing and a dynamic *beta blending* coefficient that detects divergence between raw and smoothed metrics. A tiered scoring architecture with multiplicative penalty factors enforces strict memory safety guarantees while allowing aggressive CPU consolidation. Experiments on a four-node KIND cluster demonstrate (i) consistent bin-packing of CPU-heavy bursts to a single node, (ii) graceful capacity-aware spillover for memory-heavy bursts, (iii) correct beta blending response to post-deletion EMA lag, and (iv) penalty-driven redirection of memory-risky pods away from over-committed

nodes even when their request-based bin-packing score is higher.

Contents

1 Introduction

1.1 Motivation

Modern cloud-native platforms depend on Kubernetes as their primary workload orchestration layer. The scheduler is the component that decides *which node* each pod runs on, and its quality directly affects cluster cost, performance, and reliability.

The default Kubernetes scheduler uses a `LeastAllocated` or `BalancedResourceAllocation` priority plugin that spreads workloads evenly across nodes. While spreading improves fault tolerance, it creates a fundamental tension with cluster economics:

- **Resource fragmentation.** Each node holds a small slice of every workload, making it unlikely that any single node becomes fully free.
- **Wasted idle capacity.** Partially-utilized nodes cannot be returned to autoscaler pools or de-provisioned.
- **Request inflation.** Production workloads often over-provision CPU and memory requests as safety margins. A pure request-based scheduler over-estimates node load, amplifying the fragmentation problem.

These issues directly impact overall cluster resource utilization and cloud compute cost efficiency. Poor workload consolidation increases the number of actively utilized nodes, preventing effective autoscaling and reducing the ability to reclaim idle infrastructure resources.

Bin-packing addresses fragmentation by packing pods tightly onto the fewest possible nodes, leaving others empty for reclamation. However, naïve bin-packing using only static resource requests ignores actual runtime utilization and can lead to memory over-commitment and OOM kills when actual usage exceeds requests.

This study investigates the following research question: *Can a runtime-aware adaptive bin-packing scheduler improve cluster resource utilization and reduce idle compute compared to the default Kubernetes scheduling strategy, while still maintaining memory safety and scheduling stability?*

Kdapt bridges these two concerns: it applies aggressive bin-packing for CPU (which is compressible) while enforcing conservative memory safety through runtime-informed metrics and hard scoring tiers.

1.2 Research Objectives

This independent study pursues the following objectives:

1. Design and implement a custom Kubernetes scheduler plugin using the official Scheduling Framework.
2. Develop a scoring algorithm that adapts to both request-based and runtime-based utilization signals.
3. Address EMA staleness during rapid cluster state changes through a lightweight divergence-aware blending mechanism.
4. Enforce memory safety through tiered scoring without sacrificing bin-packing efficiency for CPU-dominant workloads.
5. Validate the design against the default `kube-scheduler` on a local KIND cluster with diverse realistic workload profiles.

1.3 Report Organization

Section 2 reviews the Kubernetes Scheduling Framework and resource semantics. Section 3 describes the algorithm design and iterative refinements. Section 4 covers implementation details. Section 5 presents experimental results. Section 6 discusses design tradeoffs and limitations. Section 7 outlines future directions.

2 Background

2.1 Kubernetes Scheduling Framework

The Kubernetes Scheduling Framework [1] is a plugin-based architecture that exposes well-defined extension points within the scheduling lifecycle. For each pod that requires binding to a node, the scheduler executes a *scheduling cycle* composed of the following ordered stages:

`PreFilter` → `Filter` → `Score` → `NormalizeScore` → `Reserve` → `Permit` → `PreBind` → `Bind`

Kdapt implements two extension points:

`FilterPlugin` Used to filter out nodes whose projected utilization would exceed physical capacity after placing the pod.

`ScorePlugin` Returns a floating-point score $\in [0, 1]$ for each feasible node; higher scores indicate more preferred nodes.

The `Score()` method is invoked concurrently for all feasible nodes in a single scheduling cycle — one goroutine per node — so the scoring function must be thread-safe. After `Score()` completes for all nodes, `NormalizeScore()` maps raw scores to the integer range $[0, 100]$ using relative normalization.

2.2 Resource Semantics

Understanding the asymmetry between CPU and memory is fundamental to Kdapt's design.

CPU (compressible). A CPU request reserves schedulable capacity at pod placement time but does not dedicate exclusive cores. CPU cycles are time-shared among runnable containers. If a container has no CPU limit or a generous one, it may temporarily burst beyond its request when spare cycles exist. Crucially, when demand exceeds supply, the Linux kernel *throttles* processes rather than killing them: work continues, but with higher latency.

Memory (non-compressible). A memory request also reserves schedulable capacity at placement time. However, memory is not time-shared. If a pod's actual allocation exceeds available physical memory, the kernel invokes the Out-Of-Memory (OOM) Killer, terminating the container immediately and unrecoverably. Memory pressure therefore carries a qualitatively higher risk than CPU pressure.

This asymmetry drives the two core design choices in Kdapt: (i) runtime CPU utilization is trusted aggressively to detect over-provisioning and enable tighter packing, while (ii) memory scoring remains conservative, anchored to projected request-based utilization with a hard scoring floor that prevents assignment to memory-stressed nodes.

2.3 Kubernetes Metrics Server and cAdvisor

Node-level CPU and memory usage is exposed via the Kubernetes Metrics Server, which aggregates per-container statistics from cAdvisor running on each kubelet. Metrics are refreshed at a 10-second polling interval, with an additional $\sim$20 second processing lag. Newly created pods therefore take approximately 30 seconds to appear in `kubectl top nodes` output. This inherent lag motivates the EMA smoothing and beta blending mechanisms described in Section 3.

2.4 Related Work

Google Borg. Borg [4] is the large-scale cluster management system that inspired Kubernetes. It employs a combination of request-based feasibility filtering and utilization-aware priority scoring, and has evolved bin-packing strategies over time. Kdapt draws inspiration from Borg's two-level scheduling model and its use of both request and runtime signals.

Kubernetes Default Schedulers. The `NodeResourcesFit` and `NodeResourcesBalancedAllocation` plugins in the default Kubernetes scheduler implement request-based load spreading. The optional `NodeResourcesMostAllocated` plugin implements request-based bin-packing but does not incorporate runtime metrics.

Resource-Aware Scheduling. Prior work on resource-aware scheduling (e.g., Tetris [5], Paragon [6]) has demonstrated that incorporating runtime signals significantly improves placement quality. Kdapt applies similar intuitions in the Kubernetes plugin context, with the additional requirement of real-time response to EMA staleness.

3 System Design

3.1 Design Philosophy

Kdapt's scoring function embeds three design principles:

1. **Pack first, spread second.** Prefer the node with the highest current utilization score, subject to safety constraints.
2. **Future-aware placement.** Score based on *projected* utilization after placing the pod, not current utilization.
3. **Asymmetric risk treatment.** Apply aggressive runtime-aware scoring for CPU; apply conservative safety margins for memory, backed by a hard scoring tier boundary.

3.2 Algorithm Evolution

The scoring algorithm evolved through four distinct iterations, each addressing a failure mode discovered in the prior version.

3.2.1 Stage 1 — Static Scoring

The initial implementation returned a hardcoded score of 100 for a target node and 60 for all others, serving as a controlled smoke-test to verify plugin invocation and framework correctness. All 10 replicas of the test deployment were placed on the preferred node, confirming the plugin was being called correctly.

3.2.2 Stage 2 — Request-Based Bin-Packing

The first algorithmic stage scored nodes using only their static resource requests:

$$s_{\text{req}} = 0.7 \cdot u_{\text{cpu}}^{\text{req}} + 0.3 \cdot u_{\text{mem}}^{\text{req}} \tag{1}$$

where $u_{\text{cpu}}^{\text{req}}$ = totalCPURequested/allocatableCPU and similarly for memory. This experiment confirmed bin-packing behavior: with artificial resource anchors on three nodes (40% on worker3, 30% on worker2, 10% on worker), eight new pods were correctly distributed following the snowball effect (worker3 filled first, then worker2, with worker remaining empty), exactly as the static utilization scores predicted.

3.2.3 Stage 3 — Runtime-Aware Adaptive Scoring

Stage 2 made all decisions solely on static requests. Request inflation (pods that request 3000m but use ∼100m) causes the scheduler to think a node is far busier than it is, preventing consolidation. Stage 3 incorporates live runtime metrics from the Metrics Server.

Metrics Collection. A background goroutine collects node metrics every 10 seconds and applies Exponential Moving Average (EMA) smoothing with $\alpha = 0.3$:

$$S_t = 0.3 \cdot R_t + 0.7 \cdot S_{t-1} \tag{2}$$

where R_t is the raw metric at time t and S_t is the smoothed value. The 0.3 alpha provides noise suppression while preserving responsiveness. Both raw (R) and smoothed (S) values are stored per node.

Projected Utilization. To make placement future-aware, Kdapt adds the incoming pod's resource request to the node's current requested allocations before scoring:

$$u_{\text{cpu}}^{\text{proj}} = \frac{\text{currentCPURequested} + \text{podCPURequest}}{\text{allocatableCPU}} \tag{3}$$

Mismatch-Adaptive Weights. The *mismatch* between request-based and runtime-based utilization is used to dynamically weight how much the score relies on runtime signals:

$$\Delta_{\text{cpu}} = \left| u_{\text{cpu}}^{\text{req}} - u_{\text{cpu}}^{\text{eff}} \right| \tag{4}$$

$$\Delta_{\text{mem}} = \left| u_{\text{mem}}^{\text{req}} - u_{\text{mem}}^{\text{eff}} \right| \tag{5}$$

$$\alpha_{\text{cpu}} = \text{clamp}(0.3 + 0.8 \cdot \Delta_{\text{cpu}},\ 0,\ 1) \tag{6}$$

$$\alpha_{\text{mem}} = \text{clamp}(0.1 + 0.4 \cdot \Delta_{\text{mem}},\ 0,\ 1) \tag{7}$$

CPU starts with a higher baseline runtime weight (0.3) because CPU is compressible and runtime signals are more reliable for CPU. Memory starts with a lower baseline weight (0.1) because memory is non-compressible and conservative placement is safer. The slope coefficients 0.8 and 0.4 control how fast each dimension escalates its runtime trust as mismatch grows.

Resource Scores. Per-dimension scores blend projected utilization (for bin-packing) with effective runtime utilization (for accuracy):

$$\text{cpuScore} = (1 - \alpha_{\text{cpu}}) \cdot u_{\text{cpu}}^{\text{proj}} + \alpha_{\text{cpu}} \cdot u_{\text{cpu}}^{\text{eff}} \tag{8}$$

$$\text{memScore} = (1 - \alpha_{\text{mem}}) \cdot u_{\text{mem}}^{\text{proj}} + \alpha_{\text{mem}} \cdot u_{\text{mem}}^{\text{eff}} \tag{9}$$

3.2.4 Stage 4 — Beta Blending for EMA Lag Correction

Problem. EMA smoothing introduces a convergence lag. When a high-CPU pod is deleted, the raw metric drops from, say, 2000m to 25m immediately. The smoothed value, however, decays slowly under $\alpha = 0.3$: it takes ∼90 seconds to reach within 10% of the new raw value. During that window, the scheduler sees a falsely high effective utilization, inflating the node's score, which could attract more pods onto a now-idle node rather than packing them elsewhere. The converse problem occurs during sudden spikes: the EMA lags behind, underestimating node load and potentially over-packing.

Solution: Scale-Invariant Beta Blending. A new coefficient β measures the fractional divergence between raw and smoothed metrics:

$$\beta_{\text{cpu}} = \text{clamp}\left(\frac{|u_{\text{cpu}}^{\text{raw}} - u_{\text{cpu}}^{\text{smo}}|}{\max(u_{\text{cpu}}^{\text{raw}},\ u_{\text{cpu}}^{\text{smo}}) + \varepsilon},\ 0,\ 1 \right) \tag{10}$$

with $\varepsilon = 0.01$ preventing division by zero at low utilization. The denominator normalization is *scale-invariant*: it produces the same β value whether utilization is 0.2% or 60%, enabling reliable detection at all utilization levels.

Semantic interpretation:

- $\beta \approx 0$: raw $\approx$ smoothed; EMA is tracking reality; trust the stable smoothed value.
- $\beta \to 1$: raw and smoothed have diverged significantly; EMA is stale; trust the raw value.

The effective utilization is computed as a convex combination:

$$u^{\text{eff}} = (1 - \beta) \cdot u^{\text{smo}} + \beta \cdot u^{\text{raw}} \tag{11}$$

Example. Suppose pods are deleted at t_0 and the collector last ran at $t_0 - \delta$: $u^{\text{raw}} = 0.0025$, $u^{\text{smo}} = 0.2225$. Then:

$$\beta = \frac{|0.0025 - 0.2225|}{0.2225 + 0.01} = \frac{0.220}{0.2325} \approx 0.946$$

$$u^{\text{eff}} = 0.054 \cdot 0.2225 + 0.946 \cdot 0.0025 \approx 0.014$$

The effective utilization collapses to near the raw value immediately, preventing the scheduler from continuing to treat the node as loaded.

3.2.5 Stage 5 — Tiered Scoring with Multiplicative Penalties

Problem: Penalty Ineffectiveness. An early additive penalty subtracted a fixed value from the final score when utilization exceeded a threshold. Because bin-packing naturally drives high scores, the penalty was often too small to change the scheduling decision: a heavily-packed node at 0.89 projected memory utilization still scored higher than a lightly-loaded alternative.

Three-Zone Memory Design. Kdapt partitions the score space based on projected memory utilization into three zones:

Table 1: Kdapt scoring zones based on projected memory utilization.

Zone	Range	Score Range	Behavior
1	$u^{\text{proj}}_{\text{mem}} \leq 0.75$	$[0.5,\ 1.0]$	Free bin-packing; CPU penalty if $u^{\text{proj}}_{\text{cpu}} > 0.8$
2	$0.75 < u^{\text{proj}}_{\text{mem}} < 0.90$	$[0.5,\ 1.0)$	Warning zone; both CPU & memory penalty factors applied
3	$u^{\text{proj}}_{\text{mem}} \geq 0.90$	$[0.0,\ 0.5)$	Critical zone; score hard-capped below 0.5

The invariant $[0.5, 1.0]$ for Zones 1/2 versus $[0.0, 0.5)$ for Zone 3 guarantees that any safe node (Zone 1 or 2) always outscores any critical node (Zone 3), regardless of CPU score or penalty

values. Zone 2 provides a smooth gradient of discouragement *before* the cliff, preventing a node at 0.89 from being treated identically to one at 0.50.

Multiplicative Penalty Factors. Within the safe tier, penalty factors reduce scores multiplicatively, preserving the invariant that scores remain in $[0.5, 1.0]$:

$$f_{\text{cpu}} = \begin{cases} \dfrac{u_{\text{cpu}}^{\text{proj}} - 0.8}{0.2} & u_{\text{cpu}}^{\text{proj}} > 0.8 \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

$$f_{\text{mem}} = \begin{cases} \dfrac{u_{\text{mem}}^{\text{proj}} - 0.75}{0.15} & 0.75 < u_{\text{mem}}^{\text{proj}} < 0.9 \\ 0 & \text{otherwise} \end{cases} \tag{13}$$

Final Score Formula. Let $w_{\text{cpu}} = 0.6$ and $w_{\text{mem}} = 0.4$, with $w_{\text{cpu}} + w_{\text{mem}} = 1$ to bound the penalized score to $[0, 1]$: (It is important to converge the summation of w_{cpu} and w_{mem} to 1, to keep all the derived values within the range of [0,1] for the algorithm to work.)

$$s_{\text{penalized}} = w_{\text{cpu}} \cdot \text{cpuScore} \cdot (1 - f_{\text{cpu}}) + w_{\text{mem}} \cdot \text{memScore} \cdot (1 - f_{\text{mem}}) \tag{14}$$

$$s_{\text{final}} = \begin{cases} 0.5 \cdot \text{clamp}(1 - p_{\text{mem}},\ 0,\ 1) & u_{\text{mem}}^{\text{proj}} \geq 0.9 \\ 0.5 + 0.5 \cdot \text{clamp}(s_{\text{penalized}},\ 0,\ 1) & \text{otherwise} \end{cases} \tag{15}$$

where $p_{\text{mem}} = (u_{\text{mem}}^{\text{proj}} - 0.9)/0.1$ is a linear Zone 3 severity factor. The $0.5 + 0.5 \cdot (\cdot)$ construction guarantees $s_{\text{final}} \in [0.5, 1.0]$ for all safe-zone nodes even under aggressive penalty.

Score Normalization. The `NormalizeScore()` extension maps raw scores to the framework's $[0, 100]$ integer range using relative normalization:

$$\text{score}_i^{\text{norm}} = \frac{\text{score}_i^{\text{raw}} - \min_j \text{score}_j^{\text{raw}}}{\max_j \text{score}_j^{\text{raw}} - \min_j \text{score}_j^{\text{raw}}} \times 100 \tag{16}$$

3.3 Complete Algorithm Summary

Component	Description
EMA	$S_t = 0.3R_t + 0.7S_{t-1}$; 10s polling interval
Beta-blending	$\beta = \lvert R - S\rvert/(\max(R, S) + \varepsilon)$; $u^{\text{eff}} = (1-\beta)S + \beta R$
Mismatch	$\Delta = \lvert u^{\text{req}} - u^{\text{eff}}\rvert$ per dimension
Alpha	$\alpha_{\text{cpu}} = 0.3 + 0.8\Delta_{\text{cpu}}$; $\alpha_{\text{mem}} = 0.1 + 0.4\Delta_{\text{mem}}$
Blend	$s = (1-\alpha)u^{\text{proj}} + \alpha u^{\text{eff}}$
Weights	$w_{\text{cpu}} = 0.6$, $w_{\text{mem}} = 0.4$; constraint $w_{\text{cpu}} + w_{\text{mem}} = 1$
CPU Penalty	$f_{\text{cpu}} \in [0, 1]$ linear past 80% projected CPU
Mem Penalty	$f_{\text{mem}} \in [0, 1]$ linear in $[0.75, 0.90]$ proj. mem
Tier	Score $\in [0.5, 1.0]$ if $u^{\text{proj}}_{\text{mem}} < 0.90$; $[0.0, 0.5)$ otherwise

3.4 Hyperparameter Choices and Justification

All numeric thresholds and weights in Kdapt are *heuristically chosen* based on three guiding principles: (i) CPU is compressible and memory is not [3], (ii) smoothed signals should dominate under steady load while raw signals should dominate during bursts, and (iii) scoring must remain bounded in $[0, 1]$ by construction. A formal sensitivity analysis is left as future work; this subsection documents the reasoning behind each parameter to enable reproducibility and informed tuning.

EMA smoothing factor ($\alpha = 0.3$). The exponential moving average weight of $\alpha = 0.3$ (i.e., $S_t = 0.3R_t + 0.7S_{t-1}$) is a conventional choice in time-series smoothing literature [7, 8] that trades responsiveness for noise rejection. A smaller α (< 0.2) would make the smoothed signal too slow to track genuine load shifts; a larger α (> 0.5) would collapse EMA back toward the raw value, eliminating the stability benefit. The 10-second polling interval reinforces this choice: at that cadence, a burst arriving in one sample window contributes 30% of the new smoothed value, which is damped further in subsequent windows.

Beta divergence regulariser ($\varepsilon = 0.01$). The constant ε in $\beta = \lvert R - S\rvert/(\max(R, S) + \varepsilon)$ is a numerical-stability guard against division by near-zero denominators when both raw and smoothed utilisation are close to zero. Its value is not behaviourally significant: any constant in the range $[0.001, 0.1]$ produces identical results under normal operating utilisation.

Mismatch-adaptive alpha slopes (0.8 for CPU, 0.4 for memory). When a mismatch $\Delta = |u_{\text{req}} - u_{\text{eff}}|$ is detected, the effective smoothing weight shifts toward the raw signal at a rate controlled by the slope. The CPU slope of 0.8 reflects the fact that CPU utilisation can spike rapidly and legitimately (e.g., a JIT warmup, a garbage-collection pause), so the scorer should react quickly. The memory slope of 0.4 is deliberately half as aggressive: because memory usage grows more monotonically and OOMKill risk is non-recoverable, over-reacting to a transient memory reading is costlier than under-reacting. The base offsets (0.3 for CPU, 0.1 for memory) encode the same asymmetry as prior beliefs before any mismatch is observed.

Final score weights ($w_{\text{cpu}} = 0.6$, $w_{\text{mem}} = 0.4$). The convex-combination constraint $w_{\text{cpu}} + w_{\text{mem}} = 1$ is a correctness invariant that keeps the penalised score in $[0, 1]$, which in turn bounds the final score formula by construction. The 0.6/0.4 split mirrors the compressibility asymmetry: CPU over-commitment degrades performance gracefully via throttling, whereas memory over-commitment terminates pods. The earlier Stage 2 prototype used 0.7/0.3 as cpu/mem score weights; the weight was revised to 0.6/0.4 in Stage 3 after adding explicit tiered memory zones, which partially absorbed the memory-safety role that the higher weight had served.

Penalty thresholds (0.8 for CPU; 0.75 and 0.90 for memory). These thresholds define the boundaries between scoring zones and are acknowledged as heuristic. The 0.80-utilization CPU warning threshold is consistent with common Kubernetes capacity-planning practice of leaving a 20% headroom above which throttling risk rises non-linearly [3]. The two-tier memory design—a warning zone beginning at 0.75 and a hard-redirect cliff at 0.90—provides a 15-percentage-point transition band in which packing is penalised but still permitted. The 0.75 lower bound was chosen so that a single further pod placement (roughly 0.125 of node memory on a typical workload) would not immediately push the node into the critical zone. The 0.90 upper bound leaves a 10% buffer before physical exhaustion, consistent with Linux kernel memory-pressure behaviour where reclaim activity increases sharply above $\approx 90\%$ page-cache occupancy. A sensitivity sweep over $\{0.70, 0.75, 0.80\}$ for the warning threshold and $\{0.85, 0.90, 0.95\}$ for the critical threshold is identified as future work.

Summary. Table 2 lists all hyperparameters, their values, the rationale category (principled analogy, conventional value, or empirical observation), and whether a formal sensitivity analysis has been performed.

Table 2: Kdapt hyperparameter summary.

Parameter	Value	Rationale	Sensitivity Analysis
EMA α	0.3	Conventional smoothing tradeoff between stability and responsiveness	Not performed
Beta ε	0.01	Prevents divide-by-zero and stabilizes low-utilization behavior	N/A (numerically stabilizing constant)
α_{cpu} base / slope	0.3 / 0.8	Faster adaptation due to CPU compressibility	Not performed
α_{mem} base / slope	0.1 / 0.4	Conservative adaptation for memory safety	Not performed
w_{CPU} / w_{mem}	0.6 / 0.4	Higher CPU weighting due to compressibility asymmetry	Not performed
CPU penalty threshold	0.80	Maintains headroom before saturation	Not performed
Memory warning zone	0.75–0.90	Gradual discouragement before critical memory pressure	Identified as future work
Memory critical threshold	0.90	Hard safety boundary before exhaustion risk	Identified as future work

4 Implementation

4.1 Plugin Structure

Kdapt is implemented as a Go package under `pkg/scheduler/framework/plugins/kdapt/kdapt.go` within a fork of the Kubernetes source tree. The plugin is registered with the scheduler framework in `pkg/scheduler/framework/plugins/registry.go`.

The core struct is:

```
type NodeRuntimeMetrics struct {
    CPUMilli             float64
    MemoryBytes          float64
    SmoothedCPUMilli     float64
    SmoothedMemoryBytes float64
}

type Kdapt struct {
```

```
    handle       fwk.Handle
    nodeMetrics  map[string]NodeRuntimeMetrics
    mutexLock    sync.RWMutex
}
```

Listing 1: Kdapt plugin struct and NodeRuntimeMetrics type.

4.2 Metrics Collection

A background goroutine is spawned once in `New()` and collects fresh metrics every 10 seconds via the Kubernetes Metrics API:

```
func (k *Kdapt) collectMetrics(
    ctx context.Context,
    metricsClient *metricsclient.Clientset,
) (map[string]NodeRuntimeMetrics, error) {
    const ema = 0.3
    nodeMetricsList, err := metricsClient.MetricsV1beta1().
        NodeMetricses().List(ctx, metav1.ListOptions{})
    if err != nil {
        return nil, err
    }
    updated := make(map[string]NodeRuntimeMetrics)
    k.mutexLock.RLock()
    prev := k.nodeMetrics
    k.mutexLock.RUnlock()
    for _, nm := range nodeMetricsList.Items {
        currCpu := float64(nm.Usage.Cpu().MilliValue())
        currMem := float64(nm.Usage.Memory().Value())
        p := prev[nm.Name]
        updated[nm.Name] = NodeRuntimeMetrics{
            CPUMilli:            currCpu,
            MemoryBytes:         currMem,
            SmoothedCPUMilli:    ema*currCpu + (1-ema)*p.
               SmoothedCPUMilli,
            SmoothedMemoryBytes: ema*currMem + (1-ema)*p.
               SmoothedMemoryBytes,
        }
    }
```

```
    return updated, nil
}
```

Listing 2: Metrics collection with EMA smoothing.

4.3 Scoring Function (Final Version)

The complete `Score()` function body is shown below:

```
const epsilonUtil = 0.01
runTimeCpuUtil  := clamp(rt.CPUMilli/allocCpu, 0, 1)
runTimeMemUtil  := clamp(rt.MemoryBytes/allocMem, 0, 1)
smoothedCpuUtil := clamp(rt.SmoothedCPUMilli/allocCpu, 0, 1)
smoothedMemUtil := clamp(rt.SmoothedMemoryBytes/allocMem, 0, 1)

betaCpu := clamp(
    math.Abs(runTimeCpuUtil-smoothedCpuUtil) /
    (math.Max(runTimeCpuUtil, smoothedCpuUtil) + epsilonUtil), 0, 1)
betaMem := clamp(
    math.Abs(runTimeMemUtil-smoothedMemUtil) /
    (math.Max(runTimeMemUtil, smoothedMemUtil) + epsilonUtil), 0, 1)

effectiveCpuUtil := (1-betaCpu)*smoothedCpuUtil + betaCpu*
   runTimeCpuUtil
effectiveMemUtil := (1-betaMem)*smoothedMemUtil + betaMem*
   runTimeMemUtil

cpuMismatch := clamp(math.Abs(requestedCpuUtil-effectiveCpuUtil), 0,
    1)
memMismatch := clamp(math.Abs(requestedMemUtil-effectiveMemUtil), 0,
    1)

alphaCpu := clamp(0.3+cpuMismatch*0.8, 0, 1)
alphaMem := clamp(0.1+memMismatch*0.4, 0, 1)

cpuScore := (1-alphaCpu)*projectedCpuUtil + alphaCpu*
   effectiveCpuUtil
memScore := (1-alphaMem)*projectedMemUtil + alphaMem*
   effectiveMemUtil
```

```
25
26 wCpu, wMem := 0.6, 0.4
27 cpuPenalityFactor, memPenalityFactor := 0.0, 0.0
28 if projectedCpuUtil > 0.8 {
29     cpuPenalityFactor = (projectedCpuUtil - 0.8) / 0.2
30 }
31 if projectedMemUtil > 0.75 && projectedMemUtil < 0.9 {
32     memPenalityFactor = (projectedMemUtil - 0.75) / 0.15
33 }
34
35 finalScore := 0.0
36 if projectedMemUtil >= 0.9 {
37     memPenality := (projectedMemUtil - 0.9) / 0.1
38     finalScore = 0.5 * clamp(1.0-memPenality, 0, 1)
39 } else {
40     penalisedScore := wCpu*cpuScore*(1-cpuPenalityFactor) +
41                       wMem*memScore*(1-memPenalityFactor)
42     finalScore = 0.5 + 0.5*clamp(penalisedScore, 0, 1)
43 }
44 finalScore = clamp(finalScore, 0, 1)
```

Listing 3: Kdapt Score() — final algorithm (key computation block).

4.4 NormalizeScore

```
1  var minScore int64 = 0
2  for i := range scores {
3      if scores[i].Score < minScore {
4          minScore = scores[i].Score
5      }
6  }
7  if minScore < 0 {
8      for i := range scores {
9          scores[i].Score -= minScore
10     }
11 }
12 var maxScore int64 = 0
13 for i := range scores {
14     if scores[i].Score > maxScore {
```

```
15          maxScore = scores[i].Score
16      }
17  }
18  for i := range scores {
19      if maxScore > 0 {
20          scores[i].Score = scores[i].Score * fwk.MaxNodeScore /
                maxScore
21      }
22  }
```

Listing 4: NormalizeScore with negative-shift guard.

4.5 Cluster Setup and Deployment

Experiments are conducted on a KIND (Kubernetes in Docker) cluster running on an Apple Silicon (ARM64) machine. The cluster topology is:

Table 3: KIND cluster configuration.

Node	Role	CPU (allocatable)	Memory (allocatable)
scheduler-lab-control-plane	Control plane	10,000m	8 Gi
scheduler-lab-worker	Worker	10,000m	8 Gi
scheduler-lab-worker2	Worker	10,000m	8 Gi
scheduler-lab-worker3	Worker	10,000m	8 Gi

Kdapt is built via:

```
make WHAT=cmd/kube-scheduler KUBE_BUILD_PLATFORM=linux/arm64
```

The resulting binary is packaged into a Docker image and deployed to the cluster as a `kube-system` Deployment using a custom RBAC configuration that grants the scheduler the necessary permissions to call the Metrics API. The scheduler is invoked by setting `schedulerName: kdapt-scheduler` in pod specs.

4.6 Workload Profiles

Six deployment manifests (each with 3 replicas) cover the request-vs-runtime mismatch space:

CPU stress is induced by `polinux/stress` running `stress --cpu N --timeout 36000`. Memory stress is induced by `stress --vm 1 --vm-bytes NM --vm-keep --timeout 36000`. `nginx:latest` is used for over-provisioned profiles.

Table 4: Workload profiles used in staged experiments.

Profile	CPU Req	Mem Req	Actual CPU	Actual Mem
`cpu-matched`	2000m	256 Mi	∼2000m	∼10 Mi
`over-prov-cpu`	3000m	512 Mi	∼100m	∼50 Mi
`cpu-hungry`	500m	256 Mi	∼2000m	∼10 Mi
`mem-matched`	200m	1 Gi	∼200m	∼900 Mi
`over-prov-mem`	500m	2 Gi	∼100m	∼200 Mi
`mem-hungry`	200m	512 Mi	∼200m	∼1.5 Gi

5 Evaluation

5.1 Experiment 1: Static Scoring Validation

A 10-replica deployment using `schedulerName: kdapt-scheduler` with the static scoring function (100 for `scheduler-lab-worker`, 60 for all others) placed all ten pods on the target node. This confirmed that the plugin was correctly registered, invoked, and influential in the scheduler's decision.

5.2 Experiment 2: Request-Based Bin-Packing Validation

Setup. Three anchor pods were deployed via `nodeSelector` to create a deliberate CPU utilization gradient: worker at 10%, worker2 at 30%, worker3 at 40% of allocatable CPU (10,000m per node). Eight additional pods each requesting 1000m CPU were then submitted without node affinity.

Results. Five pods were placed on worker3 and three on worker2; worker received no new pods. Table 5 shows the final request-based CPU utilization per node.

Table 5: Final CPU utilization after Stage 2 experiment (anchor pods + 8 new pods × 1000m). σ is the cross-node standard deviation.

Scheduler	worker	worker2	worker3	σ
Kdapt (Stage 2)	10%	60%	90%	33.0%

Kdapt's high σ (33.0%) is intentional — it indicates tight consolidation onto the fewest nodes. worker3 safely reached 90% CPU utilization before any pod was placed on worker2, confirming the snowball effect. Worker was freed entirely, providing one candidate node for scale-down or future workloads.

5.3 Experiment 3: CPU-Heavy Burst Scheduling

Setup. Eight pods each requesting 1000m CPU (image: nginx:latest, actual usage: ≈ 35m) were submitted simultaneously. Initial node state: all nodes idle with ≈ 35m actual CPU and ≈ 703 MB memory.

Results. All eight pods were placed on `scheduler-lab-worker`, which had marginally higher initial requested CPU (0.02) compared to worker2 (0.01) and worker3 (0.01).

Score progression on `scheduler-lab-worker` across the burst:

Pod	reqCPU	projCPU	Score	Decision Node
Pod 1	0.02	0.12	0.066	worker
Pod 2	0.12	0.22	0.097	worker
Pod 3	0.22	0.32	0.118	worker
Pod 4	0.32	0.42	0.130	worker
Pod 5	0.42	0.52	0.132	worker
Pod 6	0.52	0.62	0.124	worker
Pod 7	0.62	0.72	0.107	worker
Pod 8	0.72	0.82	0.080	worker

The score rises as the node fills (Pods 1–5), then decreases slightly (Pods 6–8) as the CPU penalty factor activates above 0.80 projected CPU. Crucially, the score never dropped below the competing nodes' scores (∼0.053), so all eight pods remained on worker.

Comparison. Under the default `kube-scheduler`, the same burst spread pods across all three worker nodes (3/3/2 distribution). Kdapt's packing left worker2 and worker3 with nearly no additional load, enabling them to serve as targets for future workloads or to be scaled down.

5.4 Experiment 4: Memory-Heavy Burst Scheduling

Setup. Eight pods each requesting 1 Gi memory (image: `nginx:latest`, actual usage: ∼703 MB) were submitted simultaneously.

Results. Seven pods were packed onto `scheduler-lab-worker` (score progressed 0.071 → 0.117 → 0.155 → 0.188 → 0.216 → 0.238 → 0.255 across placements). At the eighth pod, worker's allocatable memory was exhausted (7 Gi requested out of 8 Gi available, with system overhead occupying the rest). The Filter phase eliminated worker, and the eighth pod was

placed on worker2.

This demonstrates key behavior: memory bin-packing was successfully applied until the node's resource limit was reached

Comparison. The default kube-scheduler spread pods evenly (3/3/2 across all three workers), with no node approaching memory saturation.

5.5 Experiment 5: Staged Gradual Rollout (15s Delay)

Setup. Six deployment profiles (Table 4) were applied with 15-second inter-wave delays, using the beta-blending version of Kdapt.

Results. Waves 1–2 (`cpu-matched`, `mem-matched`): All six pods were packed onto `scheduler-lab-worker`, which had marginally higher initial requests. Worker snowballed from a score of 0.116 up to 0.293 while competing nodes held steady at ∼0.063.

Wave 3 (`over-prov-cpu`, 3000m CPU request / 100m actual): Worker's raw CPU jumped to 6012m (0.60 utilization) but smoothed was only 0.22. Beta blending responded correctly:

$$\beta = \frac{|0.60 - 0.22|}{0.60 + 0.01} \approx 0.63 \quad \Rightarrow \quad u^{\text{eff}} \approx 0.46$$

Without beta, effective utilization would have been 0.22, dramatically underestimating node load. Worker absorbed the first over-prov-cpu pod (projected CPU 0.98); the Filter blocked worker for the remaining two, which were placed on worker2.

Waves 4–5 (`over-prov-mem`, `cpu-hungry`): Bin-packing continued on worker2. Worker2 accumulated requested CPU up to 0.80–0.90 range.

Wave 6 (`mem-hungry`, 200m request / ∼1.5 Gi actual): Worker absorbed one mem-hungry pod (projected CPU would reach 1.0 — the penalty fired but worker still led on score). Worker3 received the remaining two mem-hungry pods. One mem-hungry pod on worker3 was subsequently OOMKilled because the stress command allocated 1.5 Gi against a 512 Mi request with no memory limit.

Key finding: Beta blending at Wave 3 correctly detected the 3× divergence between raw and smoothed CPU metrics and adjusted the effective utilization accordingly. The OOM at Wave 6 was not a scheduler failure: the scheduler correctly placed the pod on a node with ≤65% projected memory; the OOM arose because the pod's runtime memory allocation (1.5 Gi) exceeded its declared request (512 Mi) and no limit was set.

5.6 Experiment 6: Staged Interleaved Rollout (15s Delay)

Setup. The same profiles as Experiment 5 but in an interleaved order: `cpu-matched`, `mem-matched`, `over-prov-cpu`, `mem-hungry`, `cpu-hungry`, `over-prov-mem`. This was the first experiment running the *tiered scoring* version.

Results. **Waves 1–3** produced the expected bin-packing behavior, with all pods going to `scheduler-lab-worker`. Scores were in $[0.55, 0.66]$, consistent with Zone 1 behavior.

Wave 4 (`mem-hungry`): First pod (p2m4b) was placed on worker (score 0.615 with $u_{\text{mem}}^{\text{proj}} = 0.65$, safe zone, no memory penalty). Subsequent pods went to worker2.

Wave 5 (`cpu-hungry`): All three cpu-hungry pods went to worker2 (scores 0.589, 0.588, 0.585).

Wave 6 (`over-prov-mem`) — critical test of tiered scoring:

- **Pod jc2pf:** worker2 projected mem = 0.63 (Zone 1), score = 0.663. Worker3 score = 0.562. Pod goes to worker2.
- **Pod q42hd:** worker2 projected mem = 0.89 (Zone 2, warning zone). Memory penalty factor = 0.92. Final score drops to **0.5432**. Worker3 score = **0.5621**. **Pod redirected to worker3.**
- **Pod xf5wb:** worker2 still at same projected mem = 0.89. Score still = 0.5432. Worker3 (now with q42hd's request) scores 0.6092. **Pod again redirected to worker3.**

The full scoring log for pod q42hd:

```
pod=q42hd, node=scheduler-lab-worker2,
  projectedCpuUtil=0.90, projectedMemUtil=0.89,
  cpuPenalityFactor=0.50, memPenalityFactor=0.92,
  finalScore=0.5432

pod=q42hd, node=scheduler-lab-worker3,
  projectedCpuUtil=0.06, projectedMemUtil=0.27,
  cpuPenalityFactor=0.00, memPenalityFactor=0.00,
  finalScore=0.5621
```

Key finding: In a prior run (*staged-gradual-15*) without tiered scoring and multiplicative penalties, all three over-prov-mem pods had been placed on worker2 regardless of its projected memory state. With tiered scoring, two of the three pods were correctly redirected to worker3, demonstrating that the mechanism works as designed.

The memory penalty factor of 0.92 effectively zeroed out worker2's memory contribution to its final score, collapsing an otherwise dominant bin-packing node into a weak candidate.

5.7 Baseline Comparison Against Default kube-scheduler

Scheduler profiles compared. Three scheduler profiles are considered:

`LeastAllocated` **(default)** Scores each node by how far its current request-based utilization is *below* capacity. Minimizes the most-utilized node, producing uniform spreading.

`NodeResourcesMostAllocated` **(existing option)** Scores each node by how far its request-based utilization is *above* zero. Implements request-based bin-packing identical to Kdapt Stage 2 but with no runtime signals or memory safety.

Kdapt (this work) Runtime-aware adaptive bin-packing with EMA smoothing, beta blending, mismatch-adaptive weights, and tiered memory safety (Stages 3–5).

Pod placement distribution (burst experiments). Table 6 shows pod placement distribution and nodes freed in Experiments 3 and 4, where all three scheduler profiles were run against identical cluster state. Full utilization, variance, and latency metrics are reported in Table 8 (Section 5.8).

Table 6: Pod placement distribution across scheduler profiles. w/w2/w3 = pods placed on each worker node. "Nodes freed" counts workers that received zero test pods.

Experiment	Scheduler	w	w2	w3	Nodes freed
CPU burst 2*(8×1000m)	LeastAllocated	3	3	2	0
	Kdapt	**8**	0	0	**2**
Mem burst 2*(8×1 Gi)	LeastAllocated	2	3	3	0
	Kdapt	**7**	1	0	**1**

Kdapt safely freed 2 of 3 worker nodes in the CPU burst and 1 of 3 in the memory burst, compared to zero nodes freed under `LeastAllocated`. `NodeResourcesMostAllocated` (request-based bin-packing without runtime signals) would match Kdapt's distribution in these two experiments because the workloads use `nginx:latest` with negligible actual usage, leaving no request-vs-runtime mismatch for Kdapt's runtime logic to act on. The differentiating capability of Kdapt manifests in the staged workload experiments (Sections 5.5 and 5.6), where actual runtime diverges significantly from declared requests.

Qualitative capability comparison. Table 7 summarizes behavioral capabilities across the full experiment suite.

Table 7: Scheduler capability comparison. T = correct behavior observed or expected; F = failure mode applies.

Capability	Least Allocated	Most Allocated	Kdapt
Bin-packing (idle-node consolidation)	F	T	T
Over-provision detection (3000m req / 100m actual)	F	F	T
Under-provision detection (500m req / 2000m actual)	F	F	T
EMA lag correction (post-deletion / post-spike)	F	F	T
Memory safety under projected over-commit	F	F	T
Graceful spillover at node capacity	T	T	T

`LeastAllocated` handles capacity-based spillover correctly (via `NodeResourcesFit` filter) but provides no consolidation and no runtime awareness. `NodeResourcesMostAllocated` adds consolidation but remains purely request-based: it cannot detect over-provisioned workloads wasting reserved capacity (Experiment 5, Wave 3), does not react to EMA lag after pod termination, and has no mechanism to redirect pods away from memory-stressed nodes (Experiment 6, Wave 6). Kdapt addresses all five remaining dimensions through its staged algorithm design.

The staged experiments were not re-executed under `LeastAllocated` or `NodeResourcesMostAllocated` due to time constraints; a formal controlled benchmark is deferred to future work (Section 7).

5.8 Quantitative Metrics Summary

Table 8 consolidates the packing and scheduling performance metrics derived from experiment logs across all evaluated scenarios.

Scoring makespan is the total wall-clock time from the first to the last `Score()` log entry within a burst. Per-pod latency is scoring makespan divided by number of pods, and reflects the wall-clock cost of one complete scheduling cycle (all feasible nodes scored concurrently, winner selected).

Packing ratio and nodes freed. Kdapt achieves a packing ratio of 62.5–100% across experiments, freeing 1–2 worker nodes entirely. `LeastAllocated` holds a constant 37.5% packing ratio and frees zero nodes in all cases. A freed node is the primary operational outcome of bin-packing: it is available for reclamation by a cluster autoscaler.

Table 8: Quantitative metrics across experiments. Packing ratio = pods on most-packed node / total pods. σ_{CPU} = cross-node standard deviation of request-based CPU utilization after placement. Scoring makespan and per-pod latency are derived from scheduler log timestamps.

Exp.	Scheduler	Packing ratio	Peak CPU util	Nodes freed	σ_{CPU}	Scoring makespan	Per-pod latency
2*E2 (Stage 2)	Kdapt	62.5%	90%	1	33.0%	n/a	n/a
	LeastAllocated	37.5%	60%	0	4.7%	n/a	n/a
2*E3 (CPU burst)	Kdapt	100%	82%	2	38.2%	18.6 ms	2.3 ms
	LeastAllocated	37.5%	33%	0	4.9%	—	—
2*E4 (Mem burst)	Kdapt	87.5%	88%	1	38.7%	21.9 ms	2.7 ms
	LeastAllocated	37.5%	38%	0	5.9%	—	—

Utilization variance. σ_{CPU} under Kdapt (33–38%) is substantially higher than under `LeastAllocated` (5–6%). This is the expected signature of bin-packing: high variance means load is intentionally concentrated, not fragmented. Both schedulers yield the same mean utilization across nodes because the total placed capacity is identical.

Scoring latency. For the 8-pod CPU burst, all 24 scoring calls (8 pods × 3 feasible nodes, concurrent per pod) completed in 18.6 ms, yielding a per-pod scheduling latency of ∼2.3 ms. The memory burst completed in 21.9 ms at 2.7 ms per pod. These figures cover only the `Score()` + `NormalizeScore()` phases; end-to-end pod startup time was not instrumented and is identified as a measurement gap below.

Measurement gaps. Three standard metrics are not available from the current setup:

- **End-to-end pod startup time.** Requires timestamping from pod creation to first `Running` transition via the Kubernetes watch API.
- **Scheduler throughput.** Requires sustained admission of hundreds of pods under a controlled arrival rate; these bursts use at most 8 pods.
- **Makespan for staged workloads.** Experiments 5 and 6 use deliberate 15-second inter-wave delays, making wall-clock makespan uninformative as a scheduler metric.

Instrumenting these via a benchmarking harness is deferred to future work (Section 7).

6 Discussion

6.1 Memory Safety Through Tiered Scoring

The three-zone memory design proved effective in Experiment 6, where pods q42hd and xf5wb were correctly redirected away from a node at 89% projected memory utilization.

The key property of tiered scoring is that it *structurally guarantees* safety: the score interval separation ensures that a safe-zone node always beats a critical-zone node regardless of the bin-packing score differential. This is qualitatively stronger than additive penalties, which can always be overcome by a sufficiently large packing score advantage.

Zone 2 (the warning gradient from 75% to 90%) is essential: without it, a node at 89% is treated identically to one at 50% until it crosses the critical threshold — too late for effective redirection.

6.2 Beta Blending Effectiveness

Experiment 5 demonstrated beta blending in action at Wave 3. When worker's raw CPU spiked to 6012m against a smoothed value of 2176m, $\beta \approx 0.63$, pulling the effective utilization from 0.22 to 0.46. Without beta, the scheduler would have seen worker as far less loaded than it actually was, potentially scheduling additional pods onto it.

Beta blending also handles the post-termination case: when pods are deleted, raw metrics collapse nearly instantly, but EMA takes ∼90 seconds to converge. Beta detects this divergence and adjusts effective utilization toward the lower raw signal, preventing the scheduler from continuing to avoid an essentially-idle node.

6.3 Limitations

10-Second Polling Granularity. All pods scheduled within a single 10-second collection interval see the same runtime snapshot. During fast bursts, a node can receive multiple pods before metrics are refreshed. The projected utilization mechanism partially compensates (request-based projections update per pod), but the runtime component remains stale. Reducing the polling interval to 5 seconds would halve the worst-case staleness window but double the Metrics API call rate.

Within-Tick Staleness. Beta blending cannot detect divergence that occurs *within* a collection interval. If a pod is created at $t = 0$ and the next collection is at $t = 8$s, the

raw metric at collection time will already reflect the new pod's usage, but any scheduling decisions between $t = 0$ and $t = 8$s use the pre-creation snapshot. This is a fundamental limitation of any polling-based metrics architecture.

Integer Score Truncation. The framework normalizes scores to the integer range $[0, 100]$. Scores that differ by ≤ 0.01 (e.g., 0.5780 vs 0.5781) may truncate to the same integer (57), resulting in a tie resolved by insertion order. This is a framework limitation, not an algorithm defect, but it means that very-close scoring decisions are not guaranteed to be deterministic.

Static Hyperparameters. All thresholds ($w_{\text{cpu}} = 0.6$, $w_{\text{mem}} = 0.4$, $\alpha = 0.3$ for EMA, penalty thresholds at 0.75/0.80/0.90) are hardcoded. Different workload profiles and cluster configurations may benefit from different values. Dynamic adaptation of these parameters through reinforcement learning or online optimization is an open direction.

6.4 Correctness of Projected Utilization

Experiment 3 provided an important validation: the projected CPU utilization passed to the second pod's scoring cycle (0.12) matched exactly the *requested* CPU observed on worker in that cycle (0.12). This confirms that the request accounting maintained by the Kubernetes scheduler framework is consistent with Kdapt's projection arithmetic, and that projected utilization is a reliable forward-looking signal for penalty activation.

7 Future Work

7.1 Dynamic Hyperparameter Adaptation

The most impactful direction is replacing the static thresholds and weights with a learned policy. A lightweight online reinforcement learning agent (e.g., contextual bandit or Q-learning with a small state space) could observe scheduling outcomes (OOM events, CPU throttling, node utilization variance) and adjust w_{cpu}, w_{mem}, and the penalty thresholds in real time. The stateless nature of the per-pod scheduling decision makes this amenable to online updates.

7.2 Formal Comparison Against Default kube-scheduler

A systematic benchmarking study should compare Kdapt and the default scheduler across:

- Pod placement distribution by node
- Scheduling latency per pod (time from pod creation to binding)

- Packing tightness (nodes freed before cluster capacity is exhausted)
- OOM event rate across diverse workload mixes

7.3 Multi-Resource Generalization

Kdapt currently considers only CPU and memory. Extending the framework to other resource dimensions (GPU, network bandwidth, storage IOPS) would require a generalized multi-dimensional bin-packing formulation and corresponding penalty structures.

8 Conclusion

This report presented Kdapt, a custom Kubernetes scheduler plugin that implements runtime-aware adaptive bin-packing. Starting from a static scoring stub and evolving through five algorithmic stages, the final design fuses static resource requests with smoothed and beta-blended runtime metrics to produce a robust scheduling signal.

Four experiments on a KIND cluster demonstrated:

1. **CPU consolidation:** 8 CPU-heavy pods packed onto one node vs. 3-node spread under the default scheduler.
2. **Memory consolidation:** 7-of-8 memory-heavy pods packed onto one node, with correct spillover at capacity.
3. **Beta blending:** Correct post-spike and post-deletion EMA lag correction, with β pulling effective utilization from 0.22 to 0.46 during a CPU burst.
4. **Tiered scoring:** Memory-risky pods (projectedMem = 0.89) correctly redirected to a safer node despite having a higher raw bin-packing score, validated by the reduction of worker2's score from 0.663 to 0.543 through a memPenaltyFactor of 0.92.

Kdapt demonstrates that incorporating runtime signals, divergence-aware blending, and tiered memory safety into the Kubernetes scheduling pipeline can significantly improve cluster efficiency and resource safety compared to the default load-spreading strategy.

References

[1] Kubernetes Authors. *Scheduling Framework — Kubernetes Documentation*. `https://kubernetes.io/docs/reference/scheduling/config/#extension-points`, 2024.

[2] Kubernetes Authors. *Scheduling Plugins — Kubernetes Documentation*. `https://kubernetes.io/docs/reference/scheduling/config/#scheduling-plugins`, 2024.

[3] Kubernetes Authors. *Resource Management for Pods and Containers — Kubernetes Documentation*. `https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/`, 2024.

[4] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. *Large-scale cluster management at Google with Borg*. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015.

[5] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella. *Multi-resource packing for cluster schedulers*. In *ACM SIGCOMM Computer Communication Review*, 44(4):455–466, 2014.

[6] C. Delimitrou and C. Kozyrakis. *Paragon: QoS-aware scheduling for heterogeneous datacenters*. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013.

[7] C. C. Holt. *Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages*. ONR Research Memorandum 52, Carnegie Institute of Technology, 1957.

[8] R. G. Brown. *Exponential Smoothing for Predicting Demand*. Arthur D. Little, Cambridge, MA, 1956.

[9] Kubernetes SIGs. *Kubernetes Metrics Server*. `https://github.com/kubernetes-sigs/metrics-server`, 2024.

[10] Kubernetes SIGs. *KIND: Kubernetes in Docker*. `https://kind.sigs.k8s.io`, 2024.